Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 4
з дисципліни: «Теорія алгоритмів»
Тема: «Робота з хеш-таблицями»

Варіант № 22

Виконав:
Студент групи АІ-254
Церковний О. Г.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритмів створення і обробки хеш-таблиць.

Завдання:

1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм створення відкритої хеш-таблиці при хешуванні з ланцюжками за варіантом завдання із Таблиці «Варіанти завдань», використовуючи хеш-функції, побудовані за методом ділення $h(K) = K \bmod 13$ та множення $h(k) = [16 (kA \bmod 1)]$.

2. Виконати візуалізацію створення відкритої хеш-таблиці, надати пояснення.

3. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм створення закритої хеш-таблиці при хешуванні з відкритою адресацією за варіантом завдання із Таблиці «Варіанти завдань», використовуючи хеш-функції, побудовані за методом ділення $h(K) = K \bmod 13$ та множення $h(k) = [16 (kA \bmod 1)]$.

4. Виконати візуалізацію створення закритої хеш-таблиці, надати пояснення.

5. Порівняти ефективність алгоритмів хешування з ланцюжками та з відкритою адресацією:

− знайдіть кількість порівнянь при пошуку (видаленні) будь-якого елемента із побудованих хеш-таблиць та вкажіть елемент з максимальною кількістю порівнянь у кожній хеш-таблиці;

− підрахуйте середню кількість порівнянь для пошуку (видалення) елемента в кожній хеш-таблиці.

Варіант	Послідовність
22	Як дбаєш, так і маєш: хто землю робить, той хліб добрий робить

Результати виконання завдання:

1. Підготовка вхідних даних та обчислення хеш-значень

1.1 Визначення числових ключів

Для виконання лабораторної роботи використовується послідовність слів відповідно до варіанта №22: «Як дбаєш, так і маєш: хто землю робить, той хліб добрий робить». Послідовність містить 12 слів.

Для побудови хеш-таблиць кожне слово перетворюється на числовий ключ. Для цього використовується український алфавіт, у якому кожній літері відповідає її порядковий номер від 1 до 33 (рис. 4.1). Числовий ключ K визначається як сума порядкових номерів усіх літер слова. Розділові знаки не враховуються, усі літери переводяться у верхній регістр.

№ п.п.	Буква	№ п.п.	Буква
1	А а	18	Н н
2	Б б	19	О о
3	В в	20	П п
4	Г г	21	Р р
5	Ґ ґ	22	С с
6	Д д	23	Т т
7	Е е	24	У у
8	Є є	25	Ф ф
9	Ж ж	26	Х х
10	З з	27	Ц ц
11	И и	28	Ч ч
12	І і	29	Ш ш
13	Ї ї	30	Щ щ
14	Й й	31	Ь ь
15	К к		
16	Л л	32	Ю ю
17	М м	33	Я я

Рисунок 4.1 – Порядкові номери літер українського алфавіту

Слово «РОБИТЬ» зустрічається в послідовності двічі (8-ме та 12-те слово). Хеш-таблиця реалізує словник, у якому кожне слово зберігається лише один раз, тому перед вставкою виконується пошук: друге входження «РОБИТЬ» буде знайдене в таблиці й повторно не додається. Отже, у таблиці зберігається n = 11 унікальних слів. Число K використовується лише для обчислення хеш-адреси; однакові значення K не означають однакові слова. Наприклад, «ТОЙ» і «ХЛІБ» мають однакове K = 56, тому різних значень K лише 10, але це різні слова, і в таблиці зберігаються обидва. Під час пошуку та перевірки на повтор порівнюються самі слова, а не їхні числові ключі.

Таблиця 4.1 – Числові ключі слів варіанта №22

№	Слово	Сума номерів літер	Ключ K
1	ЯК	33+15	48
2	ДБАЄШ	6+2+1+8+29	46
3	ТАК	23+1+15	39
4	І	12	12
5	МАЄШ	17+1+8+29	55
6	ХТО	26+23+19	68
7	ЗЕМЛЮ	10+7+17+16+32	82
8	РОБИТЬ	21+19+2+11+23+31	107
9	ТОЙ	23+19+14	56
10	ХЛІБ	26+16+12+2	56
11	ДОБРИЙ	6+19+2+21+11+14	73
12	РОБИТЬ	21+19+2+11+23+31	107

1.2 Обчислення хеш-значень методами ділення та множення

Для визначення адрес розміщення елементів використовуються два методи побудови хеш-функцій. За методом ділення хеш-значення визначається як залишок від ділення ключа K на розмір таблиці m = 13. За методом множення використовується розмір таблиці m = 16 та константа Кнута A = (√5 − 1)/2 ≈ 0,6180339887.

Метод ділення: $h_1(K) = K \bmod 13$.

Наприклад, для слова «ЯК»: $h_1(48) = 48 \bmod 13 = 9$ ($48 = 3 \cdot 13 + 9$).

Метод множення: $h_2(K) = [16 \cdot \{K \cdot A\}]$, де $\{K \cdot A\}$ – дробова частина добутку K·A.

Квадратні дужки [x] позначають округлення вниз до найближчого цілого числа.

Для слова «ЯК»: 48 · 0,6180339887 = 29,6656314576; дробова частина 0,6656314576; $h_2(48) = [16 \cdot 0{,}6656314576] = [10{,}650] = 10$.

Таблиця 4.2 – Результати обчислення хеш-значень

Слово	**K**	**$h_1(K) = K \bmod 13$**	**K·A**	**$h_2(K)$, множення**
ЯК	48	9	29,6656	10

Слово	K	$h_1(K) = K \bmod 13$	K·A	$h_2(K)$, множення
ДБАЄШ	46	7	28,4296	6
ТАК	39	0	24,1033	1
І	12	12	7,4164	6
МАЄШ	55	3	33,9919	15
ХТО	68	3	42,0263	0
ЗЕМЛЮ	82	4	50,6788	10
РОБИТЬ	107	3	66,1296	2
ТОЙ	56	4	34,6099	9
ХЛІБ	56	4	34,6099	9
ДОБРИЙ	73	8	45,1165	1
РОБИТЬ	107	3	66,1296	2

За результатами таблиць 4.1 і 4.2 визначено числові ключі всіх слів варіанта №22 та обчислено їхні хеш-значення обома методами. Обидві хеш-функції дають колізії. За методом ділення в комірку 3 потрапляють «МАЄШ», «ХТО», «РОБИТЬ», а в комірку 4 – «ЗЕМЛЮ», «ТОЙ», «ХЛІБ». Слова «ТОЙ» і «ХЛІБ» мають однаковий ключ K = 56, тому вони збігаються за будь-якою хеш-функцією. За методом множення колізії виникають у комірках 1, 6, 9 і 10.

Для розв'язання колізій далі застосовуються два способи: хешування з роздільними ланцюжками та хешування з відкритою адресацією. Отримані значення використовуються для ручної побудови таблиць і перевірки програмної реалізації мовою Python.

2. Відкрите хешування з роздільними ланцюжками

Відкрите хешування (Separate Chaining) – спосіб розв'язання колізій, за якого елементи з однаковими хеш-значеннями зберігаються у зв'язаному списку (ланцюжку), прикріпленому до відповідної комірки таблиці. Слова додаються в кінець ланцюжка в порядку їх появи у вихідній послідовності;

перед додаванням ланцюжок переглядається, щоб не допустити дублювання слова.

2.1 Побудова відкритої хеш-таблиці методом ділення

Кожне слово розміщується в ланцюжку комірки, індекс якої дорівнює K mod 13 (табл. 4.2). Покроково:

− ЯК → 9; ДБАЄШ → 7; ТАК → 0; І → 12; МАЄШ → 3 – комірки були порожні, ланцюжки довжини 1;

− ХТО → 3: комірка вже містить «МАЄШ», утворюється ланцюжок МАЄШ → ХТО;

− ЗЕМЛЮ → 4 (порожня);

− РОБИТЬ → 3: додається в кінець ланцюжка МАЄШ → ХТО → РОБИТЬ;

− ТОЙ → 4: ЗЕМЛЮ → ТОЙ; ХЛІБ → 4: ЗЕМЛЮ → ТОЙ → ХЛІБ;

− ДОБРИЙ → 8 (порожня);

− РОБИТЬ (повторно) → 3: при перегляді ланцюжка слово знайдено на 3-й позиції, вставка не виконується.

Таблиця 4.3 – Відкрита хеш-таблиця за методом ділення

Індекс	Ланцюжок слів
0	ТАК
1	–
2	–
3	МАЄШ → ХТО → РОБИТЬ
4	ЗЕМЛЮ → ТОЙ → ХЛІБ
5	–
6	–
7	ДБАЄШ
8	ДОБРИЙ
9	ЯК
10	–
11	–
12	І

2.2 Побудова відкритої хеш-таблиці методом множення

Адреси обчислюються за формулою $h_2(K) = [16 \cdot \{K \cdot A\}]$, розмір таблиці – 16 комірок. Колізії виникли для слів «І» (комірка 6, вже зайнята «ДБАЄШ»), «ЗЕМЛЮ» (комірка 10, «ЯК»), «ХЛІБ» (комірка 9, «ТОЙ») і «ДОБРИЙ» (комірка 1, «ТАК»). Повторне слово «РОБИТЬ» знайдено в комірці 2 і не додається.

Таблиця 4.4 – Відкрита хеш-таблиця за методом множення

Індекс	Ланцюжок слів
0	ХТО
1	ТАК → ДОБРИЙ
2	РОБИТЬ
3	–
4	–
5	–
6	ДБАЄШ → І
7	–
8	–
9	ТОЙ → ХЛІБ
10	ЯК → ЗЕМЛЮ
11	–
12	–
13	–
14	–
15	МАЄШ

2.3 Графічна візуалізація відкритих хеш-таблиць

Для наочного представлення результатів побудовано графічні схеми: на рисунку 4.2 – таблиця за методом ділення, на рисунку 4.3 – за методом множення. Червоним кольором виділено ланцюжки, у яких виникли колізії, сірим – ланцюжки з одного елемента, порожні комірки не мають ланцюжка.

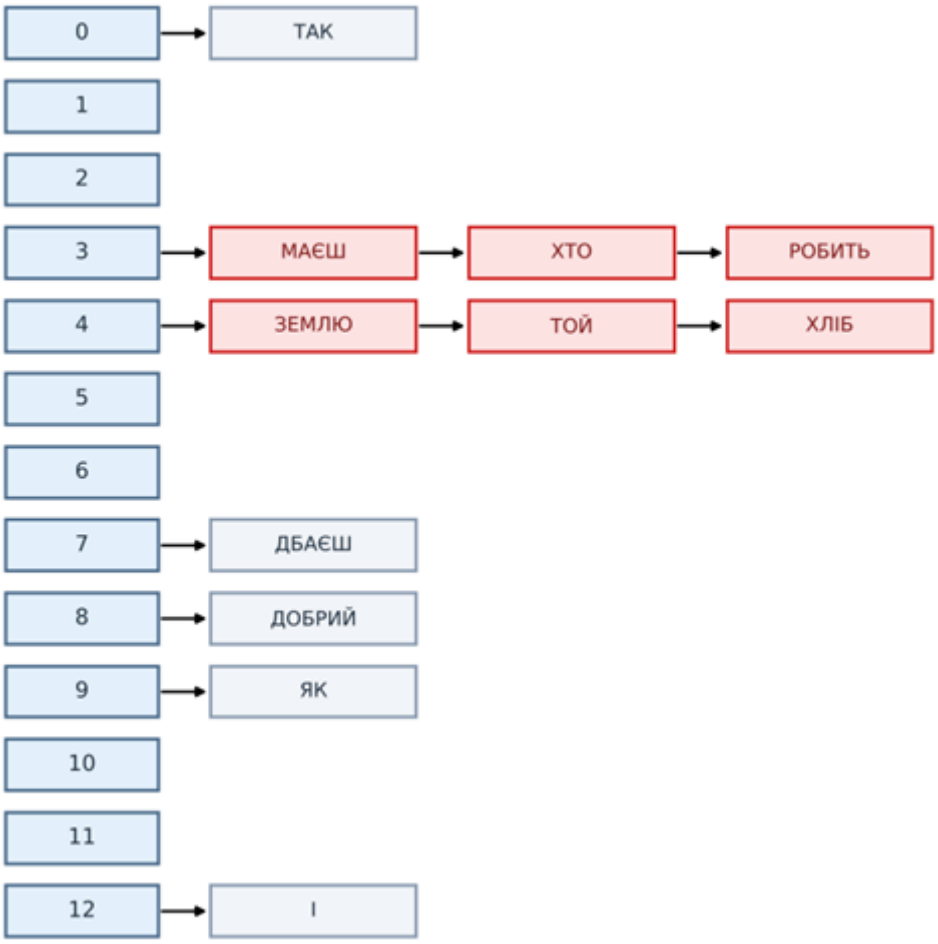


Рисунок 4.2 – Відкрита хеш-таблиця за методом ділення

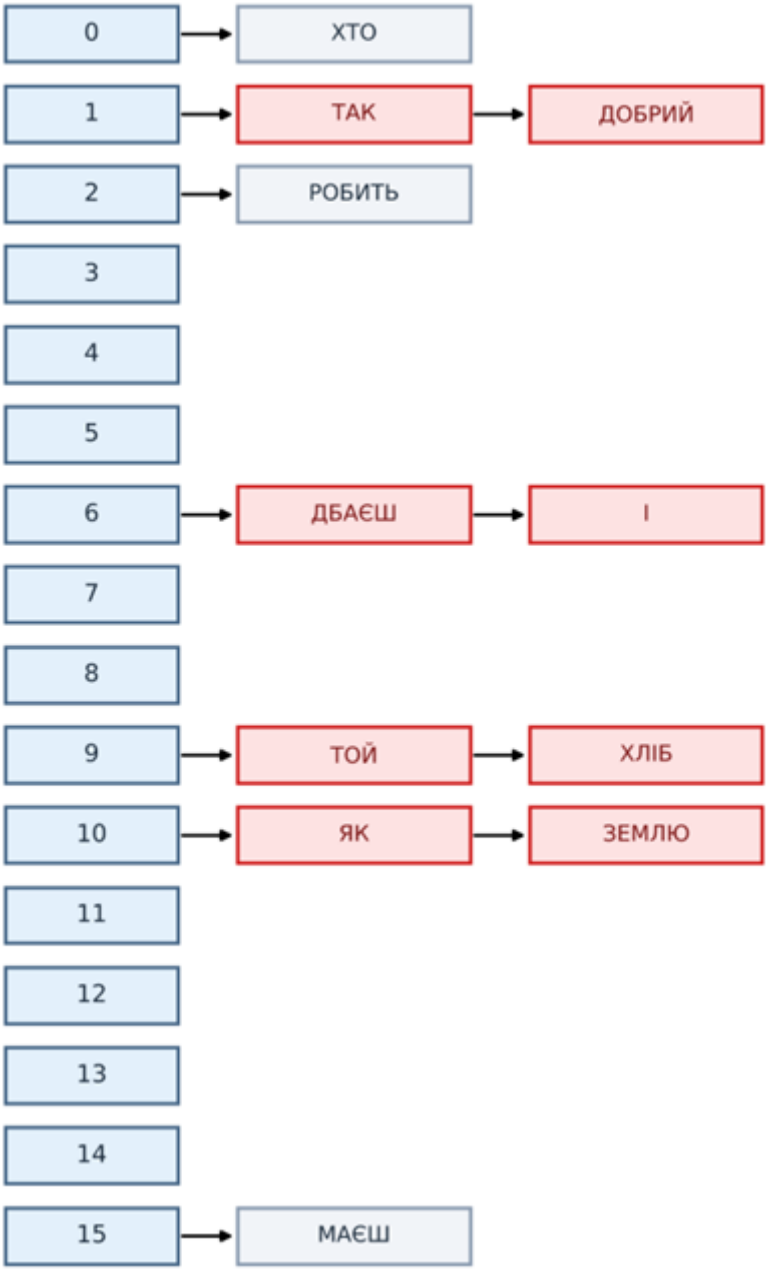


Рисунок 4.3 – Відкрита хеш-таблиця за методом множення

За методом ділення утворилися два ланцюжки по три слова (комірки 3 і 4), при цьому 6 із 13 комірок залишилися порожніми – ключі розподілено нерівномірно. За методом множення утворилися чотири ланцюжки по два слова, а максимальна довжина ланцюжка дорівнює 2, тобто розподіл рівномірніший.

2.4 Реалізація відкритого хешування мовою Python

Для перевірки ручної побудови розроблено програмну реалізацію методу роздільних ланцюжків. Функція calculate_key() обчислює числовий ключ слова, division_hash() і multiplication_hash() – хеш-адреси, build_open_hash_table() будує таблицю (з перевіркою наявності слова в ланцюжку перед вставкою), display_hash_table() виводить результат. Код наведено в Лістингу 4.1.

Лістинг 4.1 – Python-код реалізації відкритого хешування з роздільними ланцюжками

```
import math

# Розміри хеш-таблиць
M_DIV = 13
M_MUL = 16

# Константа для методу множення
A = 0.6180339887

# Послідовність слів варіанта №22
WORDS = [
    "ЯК", "ДБАЄШ", "ТАК", "І", "МАЄШ", "ХТО",
    "ЗЕМЛЮ", "РОБИТЬ", "ТОЙ", "ХЛІБ", "ДОБРИЙ", "РОБИТЬ"
]

# Український алфавіт
ALPHABET = "АБВГҐДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ"

# Словник порядкових номерів літер
LETTER_POSITIONS = {char: i + 1 for i, char in enumerate(ALPHABET)}



def calculate_key(word):
    """Обчислення числового ключа: K = сума порядкових номерів літер."""
    key = 0
    for char in word.upper():
        key += LETTER_POSITIONS.get(char, 0)
    return key



def division_hash(key):
    """Метод ділення: h(K) = K mod 13."""
    return key % M_DIV



def multiplication_hash(key):
    """Метод множення: h(K) = floor(16 * (KA mod 1))."""
```

```
    fractional_part = (key * A) % 1
    return math.floor(M_MUL * fractional_part)


def build_open_hash_table(words, m, hash_function):
    """Побудова відкритої хеш-таблиці методом роздільних ланцюжків."""
    # Ініціалізація порожніх ланцюжків
    hash_table = [[] for _ in range(m)]
    for word in words:
        key = calculate_key(word)
        address = hash_function(key)
        # У таблиці зберігаються лише унікальні слова:
        # порівнюються самі слова, а не числові ключі K
        if word in hash_table[address]:
            print(f"Слово {word} вже є в комірці {address} - пропущено")
            continue
        # Додавання слова в кінець ланцюжка
        hash_table[address].append(word)
    return hash_table


def display_hash_table(table, method):
    """Виведення хеш-таблиці."""
    print(f"\n--- {method} ---")
    for i, chain in enumerate(table):
        print(f"Індекс {i:02d}: {chain}")


# Побудова таблиці методом ділення
table_div = build_open_hash_table(WORDS, M_DIV, division_hash)
display_hash_table(table_div, "Метод ділення (M=13)")

# Побудова таблиці методом множення
table_mul = build_open_hash_table(WORDS, M_MUL, multiplication_hash)
display_hash_table(table_mul, "Метод множення (M=16)")
```

Результати виконання програми з Лістингу 4.1 наведено в таблицях 4.5 і 4.6.

Таблиця 4.5 – Результати виконання відкритого хешування за методом ділення

```
Слово РОБИТЬ вже є в комірці 3 - пропущено
--- Метод ділення (M=13) ---
Індекс 00: ['ТАК']
Індекс 01: []
Індекс 02: []
Індекс 03: ['МАЄШ', 'ХТО', 'РОБИТЬ']
Індекс 04: ['ЗЕМЛЮ', 'ТОЙ', 'ХЛІБ']
Індекс 05: []
Індекс 06: []
Індекс 07: ['ДБАЄШ']
Індекс 08: ['ДОБРИЙ']
```

```
Індекс 09: ['ЯК']
Індекс 10: []
Індекс 11: []
Індекс 12: ['І']
```

Таблиця 4.6 – Результати виконання відкритого хешування за методом множення

```
Слово РОБИТЬ вже є в комірці 2 - пропущено
--- Метод множення (M=16) ---
Індекс 00: ['ХТО']
Індекс 01: ['ТАК', 'ДОБРИЙ']
Індекс 02: ['РОБИТЬ']
Індекс 03: []
Індекс 04: []
Індекс 05: []
Індекс 06: ['ДБАЄШ', 'І']
Індекс 07: []
Індекс 08: []
Індекс 09: ['ТОЙ', 'ХЛІБ']
Індекс 10: ['ЯК', 'ЗЕМЛЮ']
Індекс 11: []
Індекс 12: []
Індекс 13: []
Індекс 14: []
Індекс 15: ['МАЄШ']
```

Аналіз результатів:

Результати ручної побудови (таблиці 4.3 і 4.4) та програмного виконання (таблиці 4.5 і 4.6) повністю збігаються: усі 11 унікальних слів розміщено у правильних комірках, повторне слово «РОБИТЬ» програма коректно виявила й пропустила.

За методом ділення виникло 4 колізії (у комірці 3 – «ХТО» і «РОБИТЬ», у комірці 4 – «ТОЙ» і «ХЛІБ»), за методом множення – також 4 колізії, але вони розподілені по чотирьох різних комірках (1, 6, 9, 10), тому ланцюжки коротші.

3. Побудова закритих хеш-таблиць із відкритою адресацією

Закрите хешування (Open Addressing) – спосіб розв'язання колізій, за якого всі елементи зберігаються безпосередньо в комірках таблиці, і кожна комірка містить не більше одного елемента. Для розв'язання колізій використовується лінійне пробування (Linear Probing): якщо комірка зайнята, перевіряється наступна; після останньої комірки перевірка продовжується з початку таблиці.

Послідовність досліджуваних комірок: $h(K, i) = (h(K) + i) \bmod m$, де $h(K)$ – початкова адреса, $i = 0, 1, 2, \ldots$ – номер спроби, m – розмір таблиці. Перед вставкою перевіряється, чи таке саме слово вже присутнє в таблиці (для повторного «РОБИТЬ»); порівнюються слова, а не числові ключі K.

3.1 Побудова закритої хеш-таблиці методом ділення

Розмір таблиці – 13 комірок, унікальних слів – 11, коефіцієнт заповнення $\alpha = 11/13 \approx 0{,}85$. Хід вставлення наведено в таблиці 4.7.

Таблиця 4.7 – Хід побудови закритої хеш-таблиці за методом ділення

№	Слово	Початкова адреса	Перевірені комірки	Кінцева адреса
1	ЯК	9	9	9
2	ДБАЄШ	7	7	7
3	ТАК	0	0	0
4	І	12	12	12
5	МАЄШ	3	3	3
6	ХТО	3	3 → 4	4
7	ЗЕМЛЮ	4	4 → 5	5
8	РОБИТЬ	3	3 → 4 → 5 → 6	6
9	ТОЙ	4	4 → 5 → 6 → 7 → 8	8
10	ХЛІБ	4	4 → 5 → 6 → 7 → 8 → 9 → 10	10
11	ДОБРИЙ	8	8 → 9 → 10 → 11	11
12	РОБИТЬ	3	3 → 4 → 5 → 6	вже є (6)

Пояснення колізій:

− Перші п'ять слів (ЯК, ДБАЄШ, ТАК, І, МАЄШ) розміщуються без колізій.

− «ХТО»: h_1 (68) = 68 mod 13 = 3, але комірка 3 зайнята «МАЄШ». H_1(68, 1) = (3 + 1) mod 13 = 4 – вільна, слово розміщується в комірці 4.

− «ЗЕМЛЮ»: h_1(82) = 4, комірка 4 тепер зайнята «ХТО» (колізія, спричинена попереднім зсувом слова «ХТО»). h_1(82, 1) = 5 – вільна.

− «РОБИТЬ»: h_1(107) = 3; комірки 3, 4, 5 зайняті, вільна – 6.

− «ТОЙ»: h_1(56) = 4; комірки 4, 5, 6, 7 зайняті (7 – «ДБАЄШ»), вільна – 8.

− «ХЛІБ»: h_1(56) = 4; комірки 4–9 зайняті (9 – «ЯК»), вільна – 10. Знадобилося 7 перевірок.

− «ДОБРИЙ»: h_1(73) = 8; комірки 8, 9, 10 зайняті, вільна – 11.

− Повторне «РОБИТЬ»: перевіряються комірки 3, 4, 5, 6; у комірці 6 знайдено таке саме слово, вставка не виконується.

Таблиця 4.8 – Закрита хеш-таблиця за методом ділення

Індекс	Слово
0	ТАК
1	–
2	–
3	МАЄШ
4	ХТО
5	ЗЕМЛЮ
6	РОБИТЬ
7	ДБАЄШ
8	ТОЙ
9	ЯК
10	ХЛІБ
11	ДОБРИЙ
12	І

Під колізією тут розуміємо вставку унікального слова, початкова комірка якого вже зайнята; повторне «РОБИТЬ» не враховується, оскільки воно не вставляється. За методом ділення колізії виникли при 6 вставках (ХТО, ЗЕМЛЮ, РОБИТЬ, ТОЙ, ХЛІБ, ДОБРИЙ), а загальна кількість зайнятих комірок, що зустрілися під час цих вставок, становить 1 + 1 + 3 + 4 + 6 + 3 = 18. Видно ефект первинної кластеризації. Таблиця циклічна, а комірка 0 також зайнята («ТАК»), тому в підсумку утворився суцільний кластер 3 → 4 → … → 12 → 0 з 11 комірок. Кожне нове слово з адресою всередині кластера мусить пройти його до кінця, і вільними залишаються лише комірки 1 і 2.

3.2 Побудова закритої хеш-таблиці методом множення

Розмір таблиці – 16 комірок, коефіцієнт заповнення $\alpha = 11/16 \approx 0{,}69$. Хід вставлення наведено в таблиці 4.9.

Таблиця 4.9 – Хід побудови закритої хеш-таблиці за методом множення

№	Слово	Початкова адреса	Перевірені комірки	Кінцева адреса
1	ЯК	10	10	10
2	ДБАЄШ	6	6	6
3	ТАК	1	1	1
4	І	6	6 → 7	7
5	МАЄШ	15	15	15
6	ХТО	0	0	0
7	ЗЕМЛЮ	10	10 → 11	11
8	РОБИТЬ	2	2	2
9	ТОЙ	9	9	9
10	ХЛІБ	9	9 → 10 → 11 → 12	12
11	ДОБРИЙ	1	1 → 2 → 3	3
12	РОБИТЬ	2	2	вже є (2)

Пояснення колізій:

− «І»: $h_2(12) = 6$, комірка 6 зайнята «ДБАЄШ»; $h_2(12, 1) = (6 + 1) \bmod 16 = 7$ – вільна.

− «ЗЕМЛЮ»: $h_2(82) = 10$, комірка зайнята «ЯК»; $h_2(82, 1) = 11$ – вільна.

− «ХЛІБ»: $h_2(56) = 9$, збігається з «ТОЙ»; комірки 10 («ЯК») і 11 («ЗЕМЛЮ») також зайняті; вільна – 12.

− «ДОБРИЙ»: $h_2(73) = 1$, комірка зайнята «ТАК»; комірка 2 – «РОБИТЬ»; вільна – 3.

− Повторне «РОБИТЬ»: $h_2(107) = 2$, слово знайдено з першої перевірки, вставка не виконується.

Таблиця 4.10 – Закрита хеш-таблиця за методом множення

Індекс	Слово
0	ХТО
1	ТАК
2	РОБИТЬ
3	ДОБРИЙ
4	–
5	–
6	ДБАЄШ
7	І
8	–
9	ТОЙ
10	ЯК
11	ЗЕМЛЮ
12	ХЛІБ
13	–
14	–
15	МАЄШ

3.3 Аналіз результатів ручної побудови

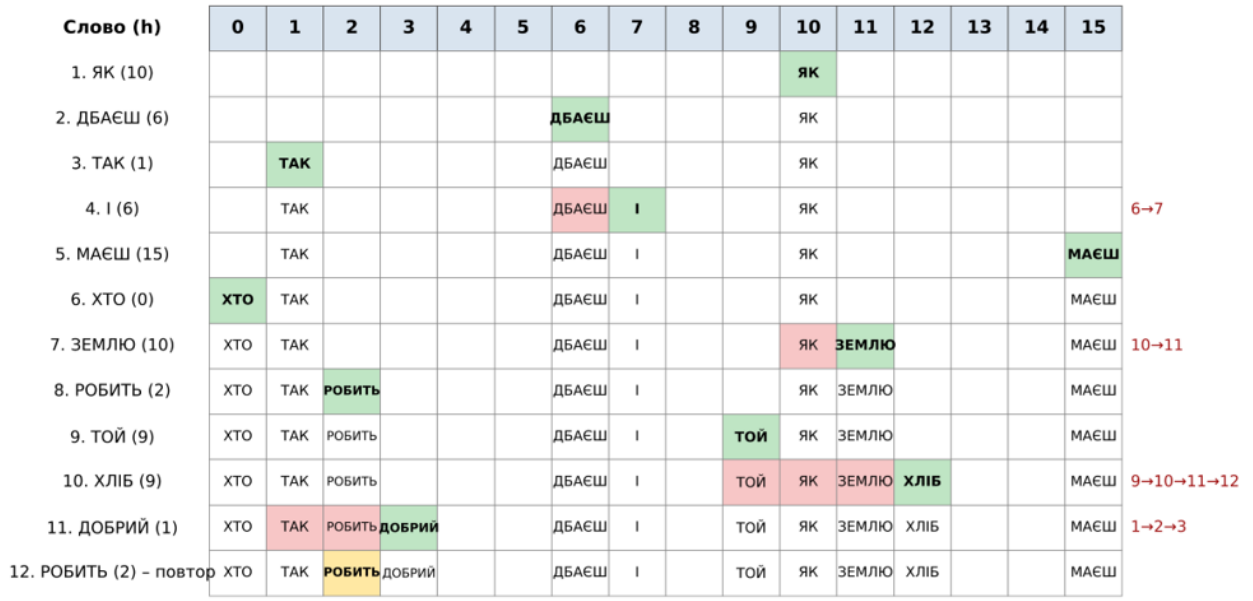
Обидві хеш-функції забезпечили розміщення всіх 11 унікальних слів без втрати елементів. За методом ділення колізії виникли при 6 вставках, і під час них зустрілося 18 зайнятих комірок. За методом множення колізії виникли при 4 вставках (І, ЗЕМЛЮ, ХЛІБ, ДОБРИЙ), і зустрілося 1 + 1 + 3 + 2 = 7 зайнятих комірок. Повторне «РОБИТЬ» в обох випадках не враховується, бо воно не вставляється. Метод множення виявився ефективнішим, оскільки таблиця більша (16 комірок, $\alpha \approx 0{,}69$), а адреси розподілені рівномірніше, тому кластери коротші.

3.4 Графічна візуалізація закритих хеш-таблиць

На рисунках 4.4 і 4.5 показано стан таблиці після кожного кроку вставлення. Кожен рядок відповідає одному слову (в дужках – початкова хеш-адреса). Зеленим позначено комірку, в яку записано слово, рожевим – зайняті комірки, перевірені під час лінійного пробування, жовтим – комірку, де знайдено повторне слово. Праворуч наведено послідовність перевірених комірок.

Слово (h)	0	1	2	3	4	5	6	7	8	9	10	11	12	
1. ЯК (9)										**ЯК**				
2. ДБАЄШ (7)								**ДБАЄШ**		ЯК				
3. ТАК (0)	**ТАК**							ДБАЄШ		ЯК				
4. І (12)	ТАК							ДБАЄШ		ЯК			**І**	
5. МАЄШ (3)	ТАК			**МАЄШ**				ДБАЄШ		ЯК			І	
6. ХТО (3)	ТАК			МАЄШ	**ХТО**			ДБАЄШ		ЯК			І	3→4
7. ЗЕМЛЮ (4)	ТАК			МАЄШ	ХТО	**ЗЕМЛЮ**		ДБАЄШ		ЯК			І	4→5
8. РОБИТЬ (3)	ТАК			МАЄШ	ХТО	ЗЕМЛЮ	**РОБИТЬ**	ДБАЄШ		ЯК			І	3→4→5→6
9. ТОЙ (4)	ТАК			МАЄШ	ХТО	ЗЕМЛЮ	РОБИТЬ	ДБАЄШ	**ТОЙ**	ЯК			І	4→5→6→7→8
10. ХЛІБ (4)	ТАК			МАЄШ	ХТО	ЗЕМЛЮ	РОБИТЬ	ДБАЄШ	ТОЙ	ЯК	**ХЛІБ**		І	4→5→6→7→8→9→10
11. ДОБРИЙ (8)	ТАК			МАЄШ	ХТО	ЗЕМЛЮ	РОБИТЬ	ДБАЄШ	ТОЙ	ЯК	ХЛІБ	**ДОБРИЙ**	І	8→9→10→11
12. РОБИТЬ (3) – повтор	ТАК			МАЄШ	ХТО	ЗЕМЛЮ	**РОБИТЬ**	ДБАЄШ	ТОЙ	ЯК	ХЛІБ	ДОБРИЙ	І	3→4→5→6

Рисунок 4.4 – Хід побудови закритої хеш-таблиці за методом ділення



Слово (h)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
1. ЯК (10)											**ЯК**						
2. ДБАЄШ (6)							**ДБАЄШ**				ЯК						
3. ТАК (1)		**ТАК**					ДБАЄШ				ЯК						
4. І (6)		ТАК					ДБАЄШ	**І**			ЯК						6→7
5. МАЄШ (15)		ТАК					ДБАЄШ	І			ЯК					**МАЄШ**	
6. ХТО (0)	**ХТО**	ТАК					ДБАЄШ	І			ЯК					МАЄШ	
7. ЗЕМЛЮ (10)	ХТО	ТАК					ДБАЄШ	І			ЯК	**ЗЕМЛЮ**				МАЄШ	10→11
8. РОБИТЬ (2)	ХТО	ТАК	**РОБИТЬ**				ДБАЄШ	І			ЯК	ЗЕМЛЮ				МАЄШ	
9. ТОЙ (9)	ХТО	ТАК	РОБИТЬ				ДБАЄШ	І		**ТОЙ**	ЯК	ЗЕМЛЮ				МАЄШ	
10. ХЛІБ (9)	ХТО	ТАК	РОБИТЬ				ДБАЄШ	І		ТОЙ	ЯК	ЗЕМЛЮ	**ХЛІБ**			МАЄШ	9→10→11→12
11. ДОБРИЙ (1)	ХТО	ТАК	РОБИТЬ	**ДОБРИЙ**			ДБАЄШ	І		ТОЙ	ЯК	ЗЕМЛЮ	ХЛІБ			МАЄШ	1→2→3
12. РОБИТЬ (2) – повтор	ХТО	ТАК	**РОБИТЬ**	ДОБРИЙ			ДБАЄШ	І		ТОЙ	ЯК	ЗЕМЛЮ	ХЛІБ			МАЄШ	

Рисунок 4.5 – Хід побудови закритої хеш-таблиці за методом множення

З рисунка 4.4 видно, як із ростом кластера зростає кількість перевірок (наприкінці він охоплює комірки 3 → … → 12 → 0, тобто 11 комірок з урахуванням циклічності): від 2 для «ХТО» до 7 для «ХЛІБ». На рисунку 4.5 кластери коротші (6–7, 9–12 та циклічний кластер 15–0–3), тому найдовше пробування («ХЛІБ») займає лише 4 перевірки. Схеми повністю відповідають таблицям 4.7–4.10.

3.5 Реалізація закритого хешування мовою Python

Програмна реалізація хешування з відкритою адресацією використовує ті самі функції обчислення ключа та хеш-адрес. Функція build_closed_hash_table() виконує лінійне пробування, виводить послідовність перевірених комірок для кожного слова та пропускає слово, якщо воно вже є в таблиці. Код наведено в Лістингу 4.2.

Лістинг 4.2 – Python-код реалізації закритого хешування з відкритою адресацією

```
import math
```

```
# Розміри хеш-таблиць
M_DIV = 13
M_MUL = 16

# Константа методу множення
A = 0.6180339887

# Послідовність слів варіанта №22
WORDS = [
    "ЯК", "ДБАЄШ", "ТАК", "І", "МАЄШ", "ХТО",
    "ЗЕМЛЮ", "РОБИТЬ", "ТОЙ", "ХЛІБ", "ДОБРИЙ", "РОБИТЬ"
]

# Український алфавіт
ALPHABET = "АБВГҐДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ"

# Словник порядкових номерів літер
LETTER_POSITIONS = {char: i + 1 for i, char in enumerate(ALPHABET)}



def calculate_key(word):
    """Обчислення числового ключа: K = сума порядкових номерів літер."""
    key = 0
    for char in word.upper():
        key += LETTER_POSITIONS.get(char, 0)
    return key



def division_hash(key):
    """Метод ділення: h(K) = K mod 13."""
    return key % M_DIV



def multiplication_hash(key):
    """Метод множення: h(K) = floor(16 * (KA mod 1))."""
    fractional_part = (key * A) % 1
    return math.floor(M_MUL * fractional_part)



def build_closed_hash_table(words, m, hash_function):
    """Побудова закритої хеш-таблиці з лінійним пробуванням."""
    # Ініціалізація порожніх комірок
    hash_table = [None] * m
    for word in words:
        key = calculate_key(word)
        start_address = hash_function(key)
        probes = []
        # Лінійне пробування: h(K, i) = (h(K) + i) mod m
        for i in range(m):
            address = (start_address + i) % m
            probes.append(address)
            if hash_table[address] == word:
                # Слово вже є в таблиці - повторно не вставляємо
                print(f"{word}: {probes} - вже є в комірці {address}")
                break
            if hash_table[address] is None:
                hash_table[address] = word
                print(f"{word}: {probes} -> комірка {address}")
                break
        else:
```

```
            print(f"Помилка: таблиця заповнена. Не вдалося додати: {word}")
    return hash_table



def display_hash_table(table, method):
    """Виведення закритої хеш-таблиці."""
    print(f"\n--- {method} ---")
    print("Індекс | Слово")
    print("-------|----------------")
    for i, item in enumerate(table):
        value = item if item is not None else "(NULL)"
        print(f"{i:02d}     | {value}")



# Побудова таблиці методом ділення
print("Хід вставлення (метод ділення):")
table_div = build_closed_hash_table(WORDS, M_DIV, division_hash)
display_hash_table(table_div, "Метод ділення (M=13)")

# Побудова таблиці методом множення
print("\nХід вставлення (метод множення):")
table_mul = build_closed_hash_table(WORDS, M_MUL, multiplication_hash)
display_hash_table(table_mul, "Метод множення (M=16)")
```

Результати виконання програми з Лістингу 4.2 наведено в таблицях 4.11 і 4.12.

Таблиця 4.11 – Результати виконання закритого хешування за методом ділення

```
Хід вставлення (метод ділення):
ЯК: [9] -> комірка 9
ДБАЄШ: [7] -> комірка 7
ТАК: [0] -> комірка 0
І: [12] -> комірка 12
МАЄШ: [3] -> комірка 3
ХТО: [3, 4] -> комірка 4
ЗЕМЛЮ: [4, 5] -> комірка 5
РОБИТЬ: [3, 4, 5, 6] -> комірка 6
ТОЙ: [4, 5, 6, 7, 8] -> комірка 8
ХЛІБ: [4, 5, 6, 7, 8, 9, 10] -> комірка 10
ДОБРИЙ: [8, 9, 10, 11] -> комірка 11
РОБИТЬ: [3, 4, 5, 6] - вже є в комірці 6
--- Метод ділення (M=13) ---
Індекс | Слово
-------|----------------
00     | ТАК
01     | (NULL)
02     | (NULL)
03     | МАЄШ
04     | ХТО
05     | ЗЕМЛЮ
06     | РОБИТЬ
07     | ДБАЄШ
```

```
08      | ТОЙ
09      | ЯК
10      | ХЛІБ
11      | ДОБРИЙ
12      | І
```

Таблиця 4.12 – Результати виконання закритого хешування за методом множення

```
Хід вставлення (метод множення):
ЯК: [10] -> комірка 10
ДБАЄШ: [6] -> комірка 6
ТАК: [1] -> комірка 1
І: [6, 7] -> комірка 7
МАЄШ: [15] -> комірка 15
ХТО: [0] -> комірка 0
ЗЕМЛЮ: [10, 11] -> комірка 11
РОБИТЬ: [2] -> комірка 2
ТОЙ: [9] -> комірка 9
ХЛІБ: [9, 10, 11, 12] -> комірка 12
ДОБРИЙ: [1, 2, 3] -> комірка 3
РОБИТЬ: [2] - вже є в комірці 2
--- Метод множення (M=16) ---
Індекс | Слово
-------|----------------
00      | ХТО
01      | ТАК
02      | РОБИТЬ
03      | ДОБРИЙ
04      | (NULL)
05      | (NULL)
06      | ДБАЄШ
07      | І
08      | (NULL)
09      | ТОЙ
10      | ЯК
11      | ЗЕМЛЮ
12      | ХЛІБ
13      | (NULL)
14      | (NULL)
15      | МАЄШ
```

Аналіз результатів:

Результати ручної побудови закритих хеш-таблиць (таблиці 4.8 і 4.10) та програмного виконання (таблиці 4.11 і 4.12) повністю збігаються, включно з послідовностями перевірених комірок. Це підтверджує правильність реалізації хешування з відкритою адресацією та лінійним пробуванням.

4. Порівняльний аналіз ефективності хешування

Для кожної з чотирьох хеш-таблиць визначено кількість порівнянь, необхідних для успішного пошуку (а отже й видалення, яке починається з пошуку) кожного з 11 слів. Одним порівнянням вважається перевірка шуканого слова на рівність слову в поточній комірці або елементі ланцюжка. Обчислення хеш-функції до кількості порівнянь не включається.

4.1 Визначення кількості порівнянь

При пошуку з роздільними ланцюжками кількість порівнянь дорівнює позиції слова в ланцюжку. При відкритій адресації – кількості перевірених комірок, починаючи з початкової хеш-адреси (вона збігається з кількістю перевірок під час вставлення).

Наприклад, для слова «ХЛІБ» за методом ділення:

− роздільні ланцюжки: комірка 4 містить ЗЕМЛЮ → ТОЙ → ХЛІБ, потрібно 3 порівняння;

− відкрита адресація: перевіряються комірки 4 (ХТО), 5 (ЗЕМЛЮ), 6 (РОБИТЬ), 7 (ДБАЄШ), 8 (ТОЙ), 9 (ЯК), 10 (ХЛІБ) – 7 порівнянь.

Таблиця 4.13 – Кількість порівнянь при пошуку в хеш-таблицях

Слово	Ланцюжки, ділення	Ланцюжки, множення	Адресація, ділення	Адресація, множення
ЯК	1	1	1	1
ДБАЄШ	1	1	1	1
ТАК	1	1	1	1
І	1	2	1	2
МАЄШ	1	1	1	1
ХТО	2	1	2	1

Слово	Ланцюжки, ділення	Ланцюжки, множення	Адресація, ділення	Адресація, множення
ЗЕМЛЮ	1	2	2	2
РОБИТЬ	3	1	4	1
ТОЙ	2	1	5	1
ХЛІБ	3	2	7	4
ДОБРИЙ	1	2	4	3
Усього	17	15	29	18

4.2 Обчислення середньої кількості порівнянь

Середня кількість порівнянь при успішному пошуку: $Ccep = (C_1 + C_2 + \ldots + C_n) / n$, де C_i – кількість порівнянь для пошуку i-го слова, $n = 11$ – кількість унікальних слів.

1. Роздільні ланцюжки, метод ділення: $Ccep1 = 17/11 \approx 1{,}55$
2. Роздільні ланцюжки, метод множення: $Ccep2 = 15/11 \approx 1{,}36$
3. Відкрита адресація, метод ділення: $Ccep3 = 29/11 \approx 2{,}64$
4. Відкрита адресація, метод множення: $Ccep4 = 18/11 \approx 1{,}64$

Для порівняння з теорією можна скористатися наближеними оцінками середньої кількості порівнянь при успішному пошуку: $1 + \alpha/2$ для ланцюжків і $\frac{1}{2}(1 + 1/(1 - \alpha))$ для лінійного пробування (асимптотична оцінка Кнута, справедлива для великих таблиць). Для $\alpha = 11/13$ це $\approx 1{,}42$ і $\approx 3{,}75$, для $\alpha = 11/16$ – $\approx 1{,}34$ і $\approx 2{,}10$. Для ланцюжків у моделі рівномірного хешування при рівноймовірному пошуку серед n елементів точне математичне сподівання дорівнює $1 + (n - 1)/(2m)$: для $m = 13$ це $1 + 10/26 \approx 1{,}38$, для $m = 16$ це $1 + 10/32 \approx 1{,}31$. Фактичні значення (1,55; 1,36; 2,64; 1,64) мають той самий порядок і ту саму тенденцію: відкрита адресація значно чутливіша до коефіцієнта заповнення. Відхилення від оцінок пояснюються малим обсягом даних і нерівномірним розподілом конкретних ключів.

4.3 Порівняння отриманих результатів

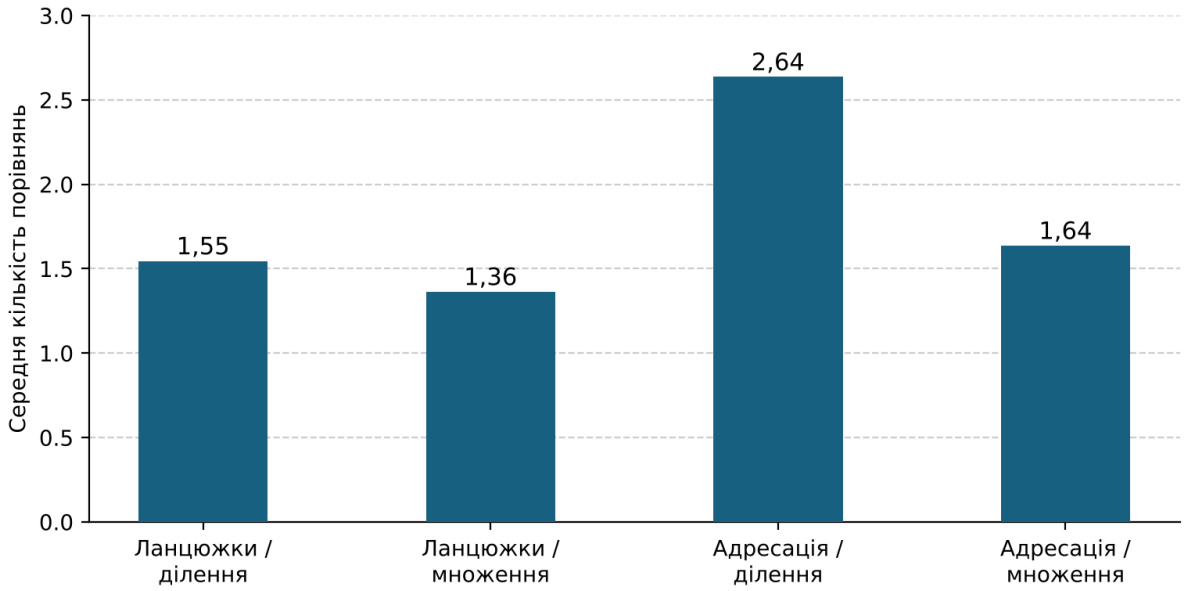


Рисунок 4.6 – Середня кількість порівнянь при успішному пошуку елементів у хеш-таблицях

Таблиця 4.14 – Порівняння ефективності методів хешування

Метод хешування	Максимум порівнянь	Елемент(и) з максимумом	Середня кількість
Ланцюжки, ділення	3	РОБИТЬ, ХЛІБ	1,55
Ланцюжки, множення	2	І, ЗЕМЛЮ, ХЛІБ, ДОБРИЙ	1,36
Адресація, ділення	7	ХЛІБ	2,64
Адресація, множення	4	ХЛІБ	1,64

Аналіз результатів:

Для послідовності варіанта №22 найменшу середню кількість порівнянь забезпечило хешування з роздільними ланцюжками за методом множення – 1,36. Ланцюжки за методом ділення дали 1,55, відкрита адресація за методом множення – 1,64, а найгірший результат показала відкрита адресація за методом ділення – 2,64.

Максимальну кількість порівнянь у таблицях з ланцюжками за методом ділення потребують слова «РОБИТЬ» і «ХЛІБ» (3 порівняння – кінці найдовших ланцюжків). За методом множення максимум дорівнює 2 (слова «І», «ЗЕМЛЮ», «ХЛІБ», «ДОБРИЙ»). У таблицях з відкритою адресацією найдорожчим є пошук слова «ХЛІБ»: 7 порівнянь за методом ділення та 4 – за методом множення.

Метод множення виявився кращим за метод ділення для обох способів розв'язання колізій. Це пояснюється тим, що хеш-функція множення розподілила ключі рівномірніше, а таблиця на 16 комірок має менший коефіцієнт заповнення. Для відкритої адресації різниця особливо помітна через первинну кластеризацію при лінійному пробуванні.

У середньому пошук у хеш-таблиці за рівномірного розподілу ключів виконується за час O(1), але в найгіршому випадку (усі ключі в одному ланцюжку або одному кластері) складність зростає до O(n). Слова «ТОЙ» і «ХЛІБ» мають однаковий ключ K = 56 і тому завжди конфліктують – це показує, що якість хешування залежить не лише від хеш-функції, а й від способу перетворення слова на числовий ключ.

Висновок:

Під час виконання лабораторної роботи досліджено, реалізовано та протестовано алгоритми створення й обробки хеш-таблиць із використанням роздільних ланцюжків (Separate Chaining) та відкритої адресації (Open Addressing) з лінійним пробуванням. Для послідовності варіанта №22 (12 слів, з них 11 унікальних) визначено числові ключі та обчислено хеш-значення методами ділення (m = 13) і множення (m = 16, A ≈ 0,618).

Шляхом ручної побудови та графічної візуалізації досліджено розміщення елементів у чотирьох хеш-таблицях і способи розв'язання колізій. Розроблено програмні реалізації мовою Python; результати їх виконання

повністю збіглися з ручною побудовою. Повторне слово «РОБИТЬ» коректно виявляється пошуком і не дублюється в таблицях.

Порівняльний аналіз показав, що найменшу середню кількість порівнянь при успішному пошуку забезпечили ланцюжки за методом множення (1,36), далі – ланцюжки за методом ділення (1,55), відкрита адресація за методом множення (1,64) та відкрита адресація за методом ділення (2,64). Максимальна кількість порівнянь становила відповідно 2, 3, 4 і 7; найдорожчим для пошуку в більшості таблиць є слово «ХЛІБ».

Отже, ефективність хешування залежить від вибору хеш-функції, коефіцієнта заповнення таблиці та способу розв'язання колізій: метод ланцюжків стійкіший до високого заповнення, а відкрита адресація з лінійним пробуванням економить пам'ять, але страждає від кластеризації при α, близькому до 1.

Посилання на GitHub: https://github.com/TserkovniyAlex/ta-lab4-22.git